

Processes

Lecture 3

Processes

- Process and threads
- Context switching
- Blocking and waking up
- Process context

Process

groups together resources

- An address space
- One or more threads
- Opened files
- Sockets
- Semaphores
- Shared memory regions
- Timers
- Signal handlers

Many other resources and status information, all stored in the **Process Control Block (PCB)**

Threads

A thread is the basic unit that the kernel process scheduler uses to allow applications to run the CPU. A thread has the following characteristics:

- Each thread has its own stack and together with the register values it determines the thread execution state
- A thread runs in the context of a process and all threads in the same process share the resources
- The kernel schedules threads not processes and user-level threads (e.g. fibers, coroutines, etc.) are not visible at the kernel level

Processes in Tock

Bibliography

for this section

Alexandru Radovici, Ioana Culic, *Getting Started with Secure Embedded Systems*

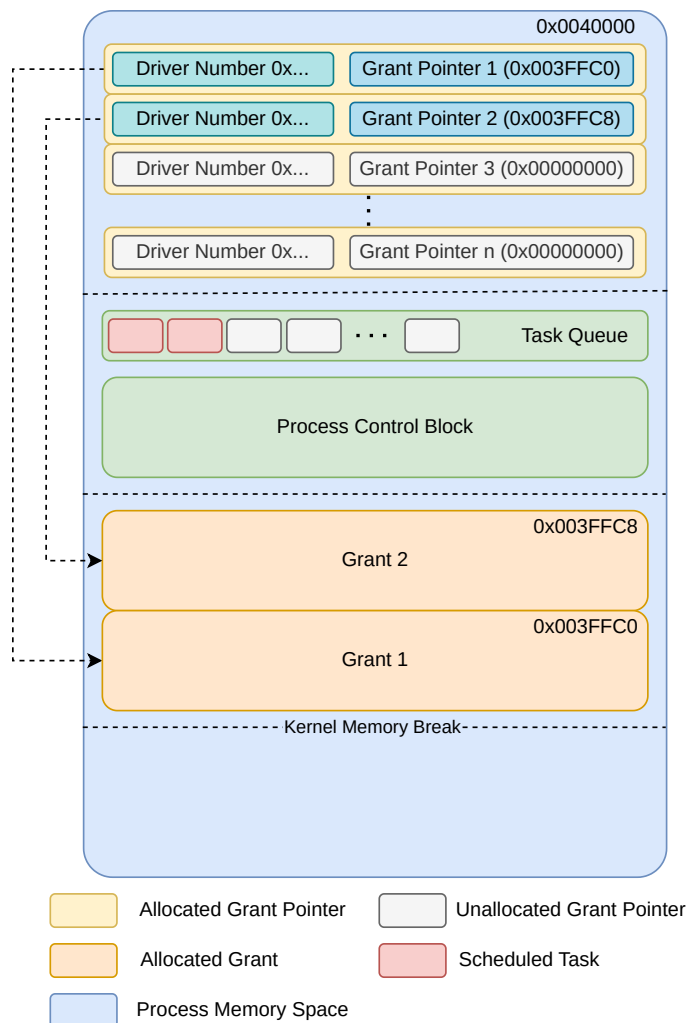
- Chapter 3 - *The Tock system architecture*

Process Control Block

information that the kernel keeps about the processes

- the actual *Process Control Block*
- the Task Queue
- Grants pointers
- Grants Data

The PCB is represented by the `Process` trait.



Process trait

identification

```
1  pub trait Process {
2      fn processid(&self) -> ProcessId;
3
4      /// Returns the ['ShortId'] generated by the application binary checker at loading.
5      fn short_app_id(&self) -> ShortId;
6
7      /// Returns the version number of the binary in this process, as specified
8      /// in a TBF Program Header. If the binary has no version assigned this
9      /// returns ['None'].
10     fn binary_version(&self) -> Option<BinaryVersion>;
11
12     /// Return the credential which the credential checker approved if the
13     /// credential checker approved a credential. If the process was allowed to
14     /// run without credentials, return 'None'.
15     fn get_credential(&self) -> Option<AcceptedCredential>;
16
17     /// Returns how many times this process has been restarted.
18     fn get_restart_count(&self) -> usize;
19
20     /// Get the name of the process. Used for IPC.
21     fn get_process_name(&self) -> &'static str;
22 }
```


Process trait

tasks

```
1  pub trait Process {
2      /// Return if there are any Tasks (upcalls/IPC requests) enqueued for the process.
3      fn has_tasks(&self) -> bool;
4
5      /// Returns the number of pending tasks.
6      fn pending_tasks(&self) -> usize;
7
8      /// Queue a [`Task`] for the process. This will be added to a per-process buffer and executed by the
9      /// scheduler. [`Task`]s are some function the process should run, for example a upcall or an IPC call.
10     fn enqueue_task(&self, task: Task) -> Result<(), ErrorCode>;
11
12     /// Remove the scheduled operation from the front of the queue and return it
13     /// to be handled by the scheduler.
14     fn dequeue_task(&self) -> Option<Task>;
15
16     /// Search the work queue for the first pending operation with the given
17     /// `upcall_id` and if one exists remove it from the queue.process
18     fn remove_upcall(&self, upcall_id: UpcallId) -> Option<Task>;
19
20     /// Remove all scheduled upcalls with the given `upcall_id` from the task queue.
21     /// Returns the number of removed upcalls.
```

Process trait

state

```
1  pub trait Process {
2      /// Returns the current state the process is in.
3      fn get_state(&self) -> State;
4
5      fn ready(&self) -> bool;
6      fn is_running(&self) -> bool;
7
8      fn set_yielded_state(&self);
9      fn set_yielded_for_state(&self, upcall_id: UpcallId);
10
11     fn stop(&self);
12     fn resume(&self);
13
14     fn set_fault_state(&self);
15
16     fn start(&self, cap: &dyn crate::capabilities::ProcessStartCapability);
17     fn try_restart(&self, completion_code: Option<u32>);
18
19     fn terminate(&self, completion_code: Option<u32>);
20
21     fn get_completion_code(&self) -> Option<Option<u32>>;
22 }
```

Process trait

memop

```
1  pub trait Process {
2      fn brk(&self, new_break: *const u8) -> Result<CapabilityPtr, Error>;
3      fn sbrk(&self, increment: isize) -> Result<CapabilityPtr, Error>;
4
5      fn number_writeable_flash_regions(&self) -> usize;
6
7      fn get_writeable_flash_region(&self, region_index: usize) -> (usize, usize);
8
9      fn update_stack_start_pointer(&self, stack_pointer: *const u8);
10     fn update_heap_start_pointer(&self, heap_pointer: *const u8);
11
12     fn build_readwrite_process_buffer(&self,
13         buf_start_addr: *mut u8, size: usize,
14     ) -> Result<ReadWriteProcessBuffer, ErrorCode>;
15     fn build_readonly_process_buffer(&self,
16         buf_start_addr: *const u8, size: usize,
17     ) -> Result<ReadOnlyProcessBuffer, ErrorCode>;
18
19     unsafe fn set_byte(&self, addr: *mut u8, value: u8) -> bool;
20
21     fn get_command_permissions(&self, driver_num: usize, offset: usize) -> CommandPermissions;
```

Process trait

memory protection unit

```
1  pub trait Process {
2      /// Configure the MPU to use the process's allocated regions.
3      fn setup_mpu(&self);
4
5      /// Allocate a new MPU region for the process that is at least
6      /// `min_region_size` bytes and lies within the specified stretch of
7      /// unallocated memory.
8      fn add_mpu_region(
9          &self,
10         unallocated_memory_start: *const u8,
11         unallocated_memory_size: usize,
12         min_region_size: usize,
13     ) -> Option<mpu::Region>;
14
15     /// Removes an MPU region from the process that has been previously added
16     /// with `add_mpu_region`.
17     fn remove_mpu_region(&self, region: mpu::Region) -> Result<(), ErrorCode>;
18 }
```

Process trait

grant

```
1  pub trait Process {
2      fn allocate_grant(&self, grant_num: usize, driver_num: usize, size: usize, align: usize) -> Result<(), ()>;
3      fn grant_is_allocated(&self, grant_num: usize) -> Option<bool>;
4
5      fn allocate_custom_grant(&self,
6          size: usize, align: usize
7      ) -> Result<(ProcessCustomGrantIdentifier, NonNull<u8>), ()>;
8
9      fn enter_grant(&self, grant_num: usize) -> Result<NonNull<u8>, Error>;
10     fn enter_custom_grant(&self, identifier: ProcessCustomGrantIdentifier) -> Result<*mut u8, Error>;
11
12     unsafe fn leave_grant(&self, grant_num: usize);
13
14     fn grant_allocated_count(&self) -> Option<usize>;
15
16     fn lookup_grant_from_driver_num(&self, driver_num: usize) -> Result<usize, Error>;
17 }
```

Process trait

subscribe

```
1  pub trait Process {
2      /// Verify that an upcall function pointer is within process-accessible
3      /// memory.
4      fn is_valid_upcall_function_pointer(&self, upcall_fn: *const ()) -> bool;
5
6      /// Set the return value the process should see when it begins executing
7      /// again after the syscall.
8      fn set_syscall_return_value(&self, return_value: SyscallReturn);
9  }
```

Process trait

context switch

```
1  pub trait Process {
2      fn set_process_function(&self, callback: FunctionCall);
3
4      /// Set the function that is to be executed when the process is resumed.
5      fn switch_to(&self) -> Option<syscall::ContextSwitchReason>;
6
7      fn get_addresses(&self) -> ProcessAddresses;
8      fn get_sizes(&self) -> ProcessSizes;
9
10     fn get_stored_state(&self, out: &mut [u8]) -> Result<usize, ErrorCode>;
11 }
```

Process trait

debug

```
1  pub trait Process {
2      fn print_full_process(&self, writer: &mut dyn Write);
3
4      fn debug_syscall_count(&self) -> usize;
5      fn debug_dropped_upcall_count(&self) -> usize;
6      fn debug_timeslice_expiration_count(&self) -> usize;
7
8      /// Increment the number of times the process has exceeded its timeslice.
9      fn debug_timeslice_expired(&self);
10
11     /// Increment the number of times the process called a syscall and record
12     /// the last syscall that was called.
13     fn debug_syscall_called(&self, last_syscall: Syscall);
14
15     /// Return the last syscall the process called. Returns `None` if the
16     /// process has not called any syscalls or the information is unknown.
17     fn debug_syscall_last(&self) -> Option<Syscall>;
18 }
```


ProcessStandard

the reference implementation of the `Process` trait

```
1  pub struct ProcessStandard<'a, C: 'static + Chip, D: 'static + ProcessStandardDebug + Default> {
2      process_id: Cell<ProcessId>,
3      app_id: ShortId,
4
5      memory_start: *const u8,
6      memory_len: usize,
7      grant_pointers: MapCell<&'static mut [GrantPointerEntry]>,
8      kernel_memory_break: Cell<*const u8>,
9      app_break: Cell<*const u8>,
10     allow_high_water_mark: Cell<*const u8>,
11
12     flash: &'static [u8],
13
14     stored_state:
15         MapCell<<<C as Chip>::UserspaceKernelBoundary as UserspaceKernelBoundary>::StoredState>,
16
17     state: Cell<State>,
18
19     tasks: MapCell<RingBuffer<'a, Task>>,
20
21     /* ... */
22 }
```

ProcessStandard

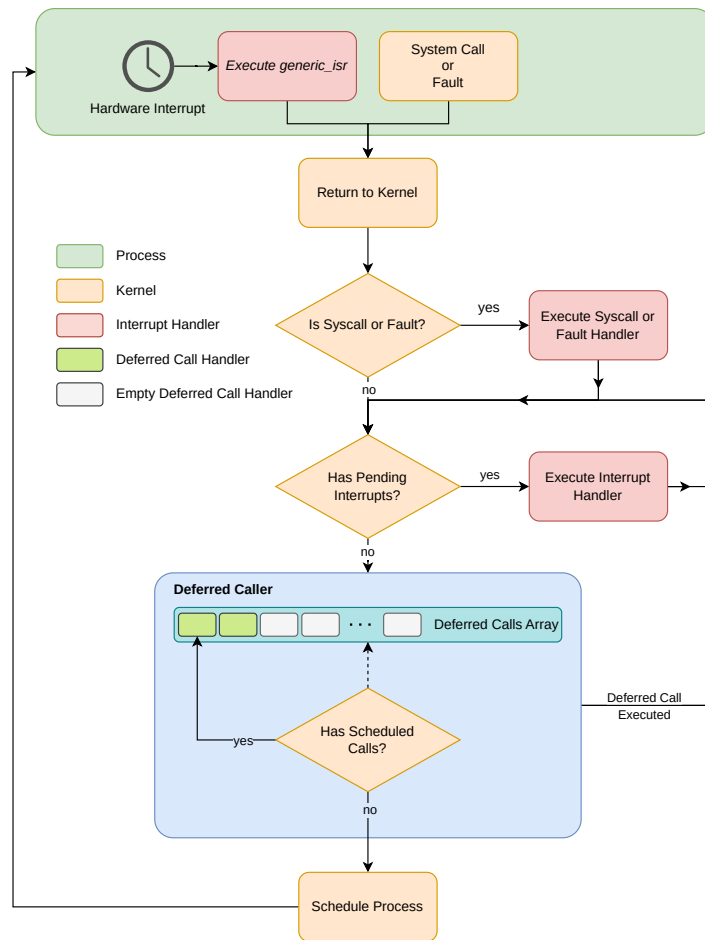
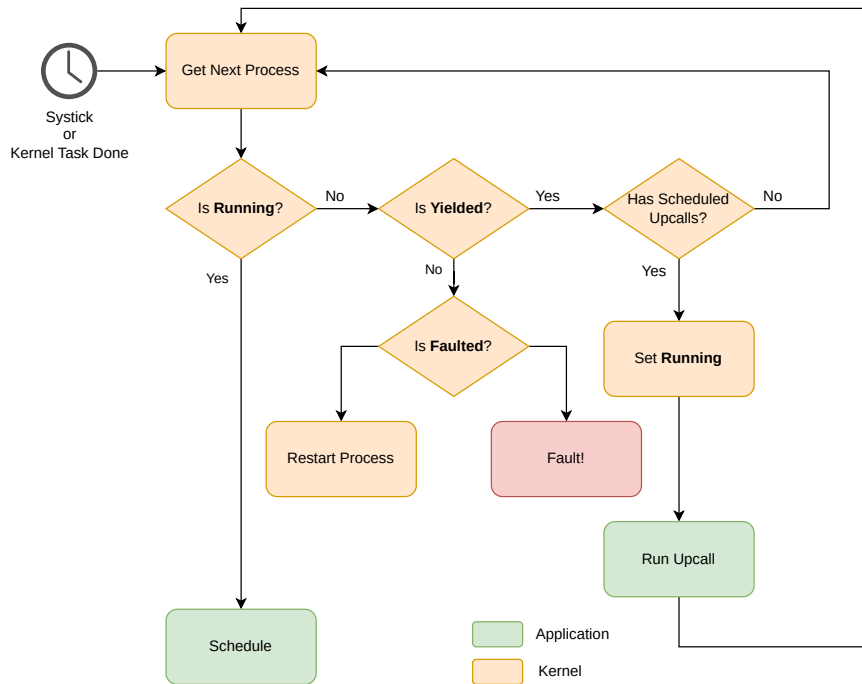
the reference implementation of the `Process` trait

```
1  pub struct ProcessStandard<'a, C: 'static + Chip, D: 'static + ProcessStandardDebug + Default> {
2      /* ... */
3      footers: &'static [u8],
4      header: tock_tbf::types::TbfHeader,
5      credential: Option<AcceptedCredential>,
6
7      kernel: &'static Kernel,
8      chip: &'static C,
9
10     fault_policy: &'a dyn ProcessFaultPolicy,
11     storage_permissions: StoragePermissions,
12     mpu_config: MapCell<<<C as Chip>::MPU as MPU>::MpuConfig>,
13     mpu_regions: [Cell<Option<mpu::Region>>; 6],
14
15
16
17     restart_count: Cell<usize>,
18     completion_code: OptionalCell<Option<u32>>,
19
20     debug: D,
21 }
```

Scheduler trait

```
1  pub trait Scheduler<C: Chip> {
2      fn next(&self) -> SchedulingDecision;
3      fn result(&self, result: StoppedExecutingReason, execution_time_us: Option<u32>);
4
5      /// Tell the scheduler to execute kernel work such as interrupt bottom
6      /// halves and dynamic deferred calls.
7      unsafe fn execute_kernel_work(&self, chip: &C) {
8          chip.service_pending_interrupts();
9          while DeferredCall::has_tasks() && !chip.has_pending_interrupts() {
10             DeferredCall::service_next_pending();
11         }
12     }
13
14     /// Ask the scheduler whether to take a break from executing userspace
15     /// processes to handle kernel tasks.
16     unsafe fn do_kernel_work_now(&self, chip: &C) -> bool {
17         chip.has_pending_interrupts() || DeferredCall::has_tasks()
18     }
19
20     /// Once a process is scheduled the kernel will try to execute it until it
21     /// has no more work to do or exhausts its timeslice.
22     unsafe fn continue_process(&self, _id: ProcessId, chip: &C) -> bool {
23         !(chip.has_pending_interrupts() || DeferredCall::has_tasks())
24     }
25 }
```

Default Scheduling



Processes in Linux

Bibliography

for this section

Daniel P. BOVET & Marco CESATI, *Understanding the LINUX KERNEL*, 3rd Edition, O'Reilly

- Chapter 3, Processes

procfs

```
+-----+
| dr-x----- 2 tavi tavi 0 2021 03 14 12:34 . |
| dr-xr-xr-x 6 tavi tavi 0 2021 03 14 12:34 .. |
| lrwx----- 1 tavi tavi 64 2021 03 14 12:34 0 -> /dev/pts/4 |
+--->| lrwx----- 1 tavi tavi 64 2021 03 14 12:34 1 -> /dev/pts/4 |
| | lrwx----- 1 tavi tavi 64 2021 03 14 12:34 2 -> /dev/pts/4 |
| | lr-x----- 1 tavi tavi 64 2021 03 14 12:34 3 -> /proc/18312/fd |
| +-----+
|
| +-----+ +-----+
$ ls -l /proc/self/ | 08048000-0804c000 r-xp 00000000 08:02 16875609 /bin/cat | Name: cat |
cmdline | 0804c000-0804d000 rw-p 00003000 08:02 16875609 /bin/cat | State: R (running) |
cwd | 0804d000-0806e000 rw-p 0804d000 00:00 0 [heap] | Tgid: 18205 |
environ | +----->| b7f46000-b7f49000 rw-p b7f46000 00:00 0 | +----->| PPid: 18133 |
exe | | b7f59000-b7f5b000 rw-p b7f59000 00:00 0 | | Uid: 1000 1000 1000 1000 |
fd +-----+ | b7f5b000-b7f77000 r-xp 00000000 08:02 11601524 /lib/ld-2.7.so | | Gid: 1000 1000 1000 1000 |
fdinfo | b7f77000-b7f79000 rw-p 0001b000 08:02 11601524 /lib/ld-2.7.so | | +-----+
maps +-----+ | bfa05000-bfa1a000 rw-p bffe000 00:00 0 [stack] | |
mem | fffffe000-ffffff000 r-xp 00000000 00:00 0 [vdso] | |
root +-----+
stat
statm
status +-----+
```

struct task_struct

```
1  struct task_struct {
2      struct thread_info thread_info;           /*      0      8 */
3      volatile long int      state;              /*      8      4 */
4      void *                  stack;             /*     12      4 */
5
6      ...
7
8      /* --- cacheline 45 boundary (2880 bytes) --- */
9      struct thread_struct thread __attribute__((__aligned__(64))); /* 2880 4288 */
10
11     /* size: 7168, cachelines: 112, members: 155 */
12     /* sum members: 7148, holes: 2, sum holes: 12 */
13     /* sum bitfield members: 7 bits, bit holes: 2, sum bit holes: 57 bits */
14     /* paddings: 1, sum paddings: 2 */
15     /* forced alignments: 6, forced holes: 2, sum forced holes: 12 */
16 } __attribute__((__aligned__(64)));
```


Threads (Windows)

how threads should look like

The typical thread implementation is one where the threads are implemented as a separate data structure which is then linked to the process data structure. For example, the *Windows* kernel uses such an implementation:

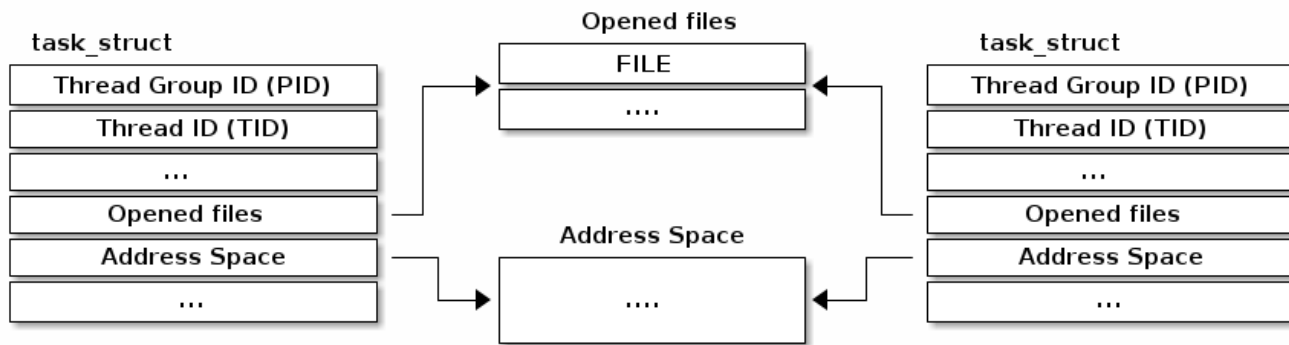
Threads

the Linux way

Linux uses a different implementation for threads. The basic unit is called a *task* (hence the `struct task_struct`) and it is used for both threads and processes.

Thus, if two threads are in the same process will point to the same resource structure instance.

If two threads are in different processes they will point to different resource structure instances.



The `clone` system call

create a process or a thread

In Linux a new thread or process is created with the `clone()` system call. Both the `fork()` system call and the `pthread_create()` function use the `clone()` implementation.

It allows the caller to decide what resources should be shared with the parent and which should be copied or isolated:

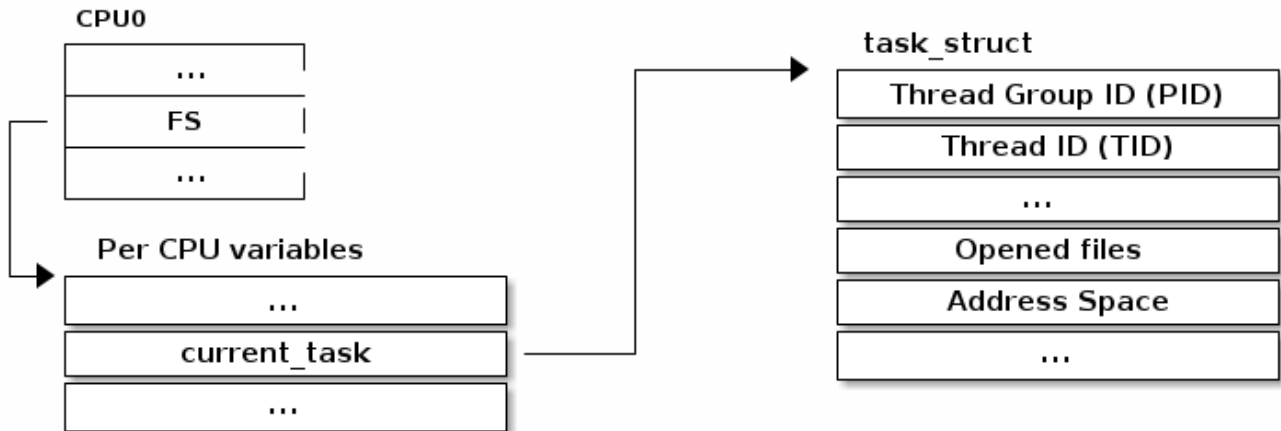
- `CLONE_FILES` - shares the file descriptor table with the parent
- `CLONE_VM` - shares the address space with the parent
- `CLONE_FS` - shares the filesystem information (root directory, current directory) with the parent
- `CLONE_NEWNS` - does not share the mount namespace with the parent
- `CLONE_NEWIPC` - does not share the IPC namespace (System V IPC objects, POSIX message queues) with the parent
- `CLONE_NEWNET` - does not share the networking namespaces (network interfaces, routing table) with the parent

Access the current `struct task_struct`

Over 90% of the system calls need to access the current process structure so it needs to be fast

- opening a file needs access to `struct task_struct`'s `file` field
- mapping a new file needs access to `struct task_struct`'s `mm` field

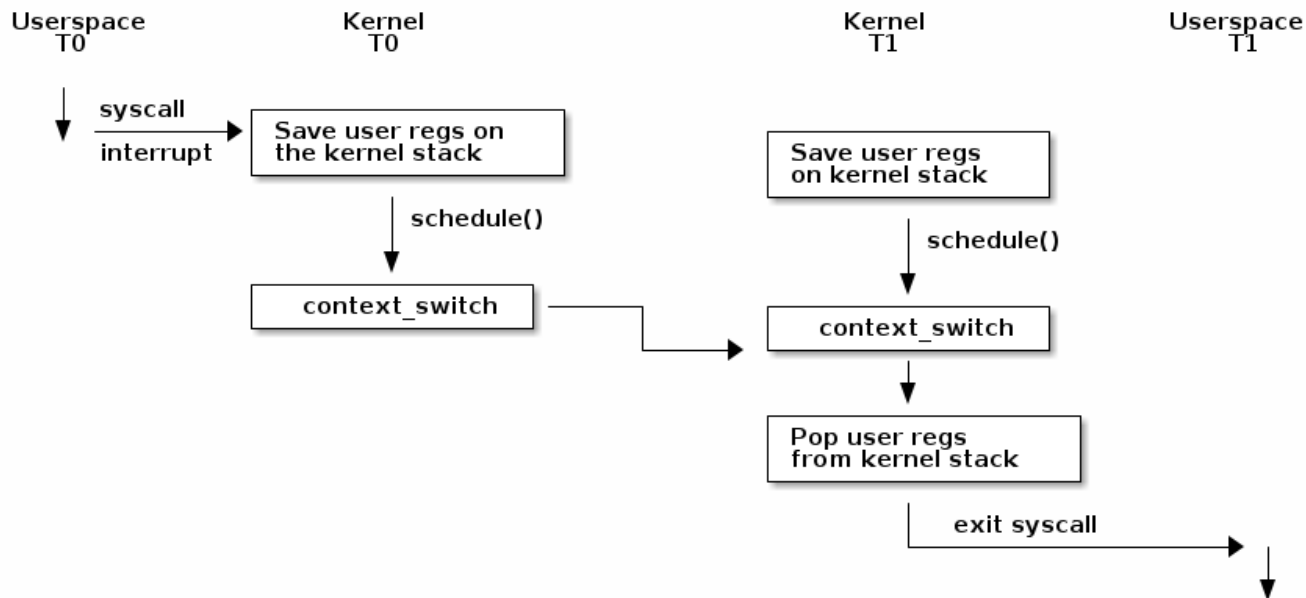
The `current` macro is available to access the current process's `struct task_struct`



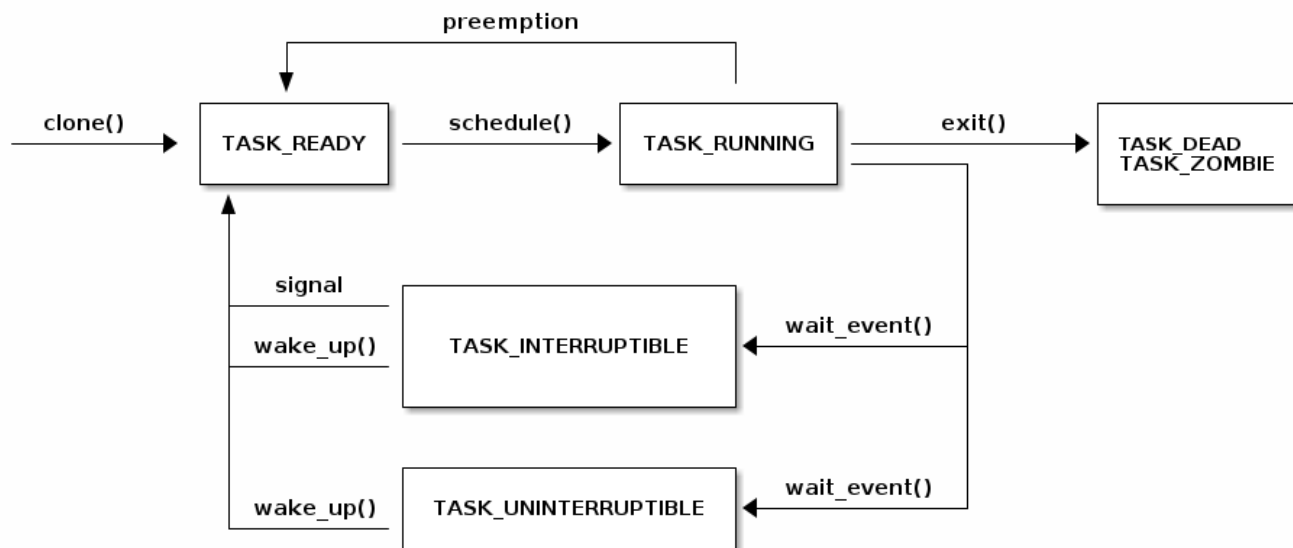
Useful process macro's

```
1  /* how to get the current stack pointer from C */
2  register unsigned long current_stack_pointer asm("esp") __attribute_used__;
3
4  /* how to get the thread information struct from C */
5  static inline struct thread_info *current_thread_info(void)
6  {
7      return (struct thread_info *)(current_stack_pointer & ~(THREAD_SIZE - 1));
8  }
9
10 #define current current_thread_info()->task
```

Context Switch



Task States



Conclusion

we talked about

- Process and threads
- Context switching
- Blocking and waking up
- Process context